

IMPLEMENTACIÓN DE ACCESO MIXTO PARA PERSONAL ACADÉMICO Y ADMINISTRATIVO

ENUNCIADO DEL EJERCICIO

La Universidad requiere que los trabajadores internos accedan mediante autenticación Windows, mientras que los consultores externos utilicen autenticación SQL Server. Diseñe e implemente una estrategia de autenticación mixta para la base de datos GradosTitulos que permita administrar ambos tipos de acceso.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 1 - AUTENTICACIÓN MIXTA (Windows + SQL)
-- Adaptado a la base de datos GradosTitulos ya existente
USE master;
GO

-- 1. Crear login Windows para el nuevo usuario local 'interno'
CREATE LOGIN [desktop-4uddpsa\interno] FROM WINDOWS;
GO

-- 2. Crear login SQL para consultores externos (solo si no existe)
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'ConsultorExterno')
    CREATE LOGIN ConsultorExterno WITH PASSWORD = N'Consultor!Ext', CHECK_POLICY = ON;
GO

USE GradosTitulos;
GO

-- 3. Crear usuarios de base de datos asociados a los logins
CREATE USER Interno FOR LOGIN [desktop-4uddpsa\interno];
GO
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'ConsultorExterno')
    CREATE USER ConsultorExterno FOR LOGIN ConsultorExterno;
GO

-- 4. Asignar permisos según el principio de mínimo privilegio
ALTER ROLE db_datareader ADD MEMBER Interno;
ALTER ROLE db_datawriter ADD MEMBER Interno;

GRANT SELECT ON SCHEMA::Academico TO ConsultorExterno;
GRANT SELECT ON SCHEMA::Tramite TO ConsultorExterno;
DENY SELECT ON [Tramite].[Pago] TO ConsultorExterno;
GO

-- 5. Verificar los accesos finales
SELECT p.name AS Usuario, p.type_desc AS TipoAutenticacion,
       r.name AS Rol, perm.state_desc AS EstadoPermiso,
       perm.permission_name AS Permiso,
       CASE perm.class
         WHEN 0 THEN 'Base de datos'
         WHEN 3 THEN SCHEMA_NAME(perm.major_id)
         WHEN 1 THEN OBJECT_NAME(perm.major_id)
         ELSE 'Otro'
       END AS Objetivo
FROM sys.database_principals p
LEFT JOIN sys.database_role_members drm ON p.principal_id = drm.member_principal_id
LEFT JOIN sys.database_principals r ON drm.role_principal_id = r.principal_id
LEFT JOIN sys.database_permissions perm ON p.principal_id = perm.grantee_principal_id
WHERE p.name IN ('Interno', 'ConsultorExterno')
ORDER BY Usuario, Rol;
```

JUSTIFICACIÓN TÉCNICA

- ✓ Modo de autenticación mixta permite coexistencia de Windows y SQL Authentication.
- ✓ Login Windows aprovecha la seguridad del sistema operativo.
- ✓ Login SQL con CHECK_POLICY = ON aplica políticas de contraseña del sistema.
- ✓ Interno: roles db_datareader y db_datawriter (lectura/escritura).
- ✓ Externo: solo SELECT en Academico y Tramite, con DENY explícito en Tramite.Pago.

BUENAS PRÁCTICAS

- ★ Principio de mínimo privilegio: permisos exactamente necesarios.
- ★ Uso de autenticación Windows para personal interno.
- ★ Políticas de contraseñas para logins SQL (CHECK_POLICY).
- ★ Denegación explícita para proteger información financiera.
- ★ Documentación y verificación para auditoría y portafolio.

ENUNCIADO DEL EJERCICIO

Los miembros del jurado necesitan acceder únicamente a información de tesis y sustentaciones. Configure accesos diferenciados utilizando autenticación Windows y SQL.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 2 - ACCESO DE JURADOS (Windows + SQL)
-- Uso tesis y sustentaciones
USE master;
GO

-- 1. Crear login Windows para jurados internos (requiere usuario local 'jurado')
CREATE LOGIN [desktop-4uddpsa\jurado] FROM WINDOWS;
GO

-- 2. Crear login SQL para jurados externos
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'JuradoExterno')
    CREATE LOGIN JuradoExterno WITH PASSWORD = N'Jur@d8Ext!', CHECK_POLICY = ON;
GO

USE GradosTitulos;
GO

-- 3. Crear usuarios de base de datos
CREATE USER JuradoInterno FOR LOGIN [desktop-4uddpsa\jurado];
GO
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'JuradoExterno')
    CREATE USER JuradoExterno FOR LOGIN JuradoExterno;
GO

-- 4. Otorgar permisos solo sobre tesis y sustentaciones
GRANT SELECT ON [Tramite].[TrabajoInvestigacion] TO JuradoExterno;
GRANT SELECT ON [Tramite].[DesignacionJurado] TO JuradoExterno;
GRANT SELECT ON [Tramite].[Sustentacion] TO JuradoExterno;
GRANT SELECT ON [Tramite].[EvaluacionTrabajo] TO JuradoExterno;
DENY SELECT ON [Tramite].[V_NotaFinalSustentacion] TO JuradoExterno;

-- Mismos permisos para el jurado interno
GRANT SELECT ON [Tramite].[TrabajoInvestigacion] TO JuradoInterno;
GRANT SELECT ON [Tramite].[DesignacionJurado] TO JuradoInterno;
GRANT SELECT ON [Tramite].[Sustentacion] TO JuradoInterno;
GRANT SELECT ON [Tramite].[EvaluacionTrabajo] TO JuradoInterno;
GRANT SELECT ON [Tramite].[V_NotaFinalSustentacion] TO JuradoInterno;
GO

-- 5. Denegar acceso a esquemas y tablas no relacionadas
DENY SELECT ON SCHEMA::Catalogo TO JuradoExterno;
DENY SELECT ON SCHEMA::Documento TO JuradoExterno;
DENY SELECT ON SCHEMA::Academico TO JuradoExterno;
GO

-- 6. Verificar accesos
SELECT p.name AS Usuario, perm.state_desc AS Estado,
       perm.permission_name AS Permiso,
       CASE perm.class
         WHEN 3 THEN SCHEMA_NAME(perm.major_id)
         WHEN 1 THEN OBJECT_NAME(perm.major_id)
         ELSE 'Base de datos'
       END AS Objetivo
FROM sys.database_permissions perm
JOIN sys.database_principals p ON perm.grantee_principal_id = p.principal_id
WHERE p.name IN ('JuradoExterno','JuradoInterno')
ORDER BY Usuario, Objetivo;
```

JUSTIFICACIÓN TÉCNICA

- ✓ Autenticación dual: Windows (internos) y SQL (externos).
- ✓ Mínimo privilegio granular: solo tesis y sustentaciones (tablas y vistas clave).
- ✓ DENY a nivel de esquema para bloquear información no autorizada.
- ✓ Vistas consolidadas para exponer tablas subyacentes.
- ✓ Verificación final asegura que los accesos queden correctamente limitados.

BUENAS PRÁCTICAS

- ★ Principio de mínimo privilegio.
- ★ Coexistencia segura de Windows y SQL Authentication.
- ★ Bloqueo por defecto: DENY a esquemas, solo lo necesario.
- ★ Documentación y auditoría con consulta de validación.
- ★ Nomenclatura descriptiva (JuradoInterno, JuradoExterno).

ENUNCIADO DEL EJERCICIO

La Oficina de Tecnologías de Información requiere aplicar políticas estrictas para las cuentas SQL que administran los trámites de grados y títulos.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 3 - POLÍTICAS ESTRICTAS PARA CUENTAS SQL DE TI
USE master;
GO

-- 1. Crear login SQL con políticas reforzadas
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'AdminTI')
    CREATE LOGIN AdminTI WITH PASSWORD = N'T1S3pur4Admi',
        CHECK_POLICY = ON,          -- cumplimiento del SO
        CHECK_EXPIRATION = ON;      -- expiración obligatoria
GO

-- 2. Restringir acceso a horario laboral (lunes a viernes)
CREATE OR ALTER TRIGGER RestringirHorario
ON ALL SERVER
FOR LOGON
AS
BEGIN
    IF ORIGINAL_LOGIN() = 'AdminTI' AND DATEPART(WEEKDAY, GETDATE()) IN (1, 7)
        ROLLBACK; -- cancela conexión (fines de semana)
END;
GO

USE GradosTítulos;
GO

-- 3. Crear usuario en la base de datos
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'AdminTI')
    CREATE USER AdminTI FOR LOGIN AdminTI;
GO

-- 4. Permisos sobre trámites (sin eliminación) y denegación a pagos
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO AdminTI;
DENY SELECT, INSERT, UPDATE, DELETE ON [Tramite].[Pago] TO AdminTI;
GO

-- 5. Auditoría: el sistema ya cuenta con triggers existentes
-- (TR_Auditoria_EliminarPersona, TR_Auditoria_ModificarCurso, etc.)

-- 6. Verificar configuración de seguridad del login
SELECT name AS Usuario, is_disabled AS CuentaDeshabilitada,
       is_policy_checked AS PoliticaContrasena,
       is_expiration_checked AS ExpiracionContrasena,
       create_date AS FechaCreacion
FROM sys.sql_logins WHERE name = 'AdminTI';

-- 7. Permisos efectivos del usuario en la base de datos
SELECT USER_NAME(perm.grantee_principal_id) AS Usuario,
       CASE perm.class
           WHEN 3 THEN SCHEMA_NAME(perm.major_id)
           WHEN 1 THEN OBJECT_NAME(perm.major_id)
           ELSE 'Base de datos'
       END AS Objetivo,
       perm.permission_name AS Permiso, perm.state_desc AS Estado
FROM sys.database_permissions perm
WHERE perm.grantee_principal_id = USER_ID('AdminTI');
```

JUSTIFICACIÓN TÉCNICA

- ✓ Políticas de contraseña obligatorias (complejidad y expiración).
- ✓ Restricción horaria con trigger de inicio de sesión a nivel de servidor.
- ✓ Permisos granulares: SELECT, INSERT, UPDATE en Tramite (sin DELETE).
- ✓ DENY completo sobre Tramite.Pago para proteger datos financieros.
- ✓ Lectura en Catálogo y Evaluación por soporte.
- ✓ Verificación posterior en sys.sql_logins y sys.database_permissions.

BUENAS PRÁCTICAS

- ★ Endurecimiento de cuentas SQL.
- ★ Control de acceso basado en tiempo.
- ★ Mínimo privilegio y segregación de funciones.
- ★ Aprovechamiento de auditoría existente.
- ★ Documentación y transparencia del script y verificaciones.

ENUNCIADO DEL EJERCICIO

La base de datos GradosTítulos será utilizada en producción. Configure cuentas de servicio específicas para SQL Server y SQL Server Agent.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 1 (TEMA 2) - CORREGIDO PARA ENTORNO REAL
-- Instancia por defecto (MSSQLSERVER), Agente posiblemente no instalado
USE master;
GO

-- 0. Habilitar xp_cmdshell temporalmente
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;
GO

-- 1. Verificar que las cuentas de servicio existan (creadas manualmente)
DECLARE @checkSQL NVARCHAR(4000) = 'net user svcSQL';
DECLARE @checkAgent NVARCHAR(4000) = 'net user svcAgent';
EXEC xp_cmdshell @checkSQL;
EXEC xp_cmdshell @checkAgent;
GO

-- 2. Configurar servicio del Motor (nombre exacto de la instancia)
DECLARE @svcName NVARCHAR(1000) = N'MSSQLSQL';
DECLARE @svcCmd NVARCHAR(4000) = 'sc config "' + @svcName + '"
    + ' obj= ".\svcSQL" password= "P@ssw0rdSQL"';
EXEC xp_cmdshell @svcCmd;
GO

-- 3. Configurar Agente solo si el servicio existe
DECLARE @agentQuery NVARCHAR(4000) = 'sc query SQLSERVERAGENT | find /I "STATE"';
DECLARE @result TABLE (line NVARCHAR(4000));
INSERT @result EXEC xp_cmdshell @agentQuery;
IF EXISTS (SELECT 1 FROM @result WHERE line LIKE '%RUNNING%' OR line LIKE '%STOPPED%')
BEGIN
    DECLARE @agentCmd NVARCHAR(4000) = 'sc config SQLSERVERAGENT obj= ".\svcAgent" password= "P@ssw0rdAgent"';
    EXEC xp_cmdshell @agentCmd;
END
ELSE
    PRINT 'El servicio SQL Server Agent no está instalado. Se omite su configuración.';
GO

-- 4. Deshabilitar xp_cmdshell (motor y agente)
EXEC sp_configure 'xp_cmdshell', 0; RECONFIGURE;
GO

-- 5. Validar cuentas actuales de los servicios
SELECT servicename, service_account, status_desc
FROM sys.dm_server_services
WHERE servicename IN (N'SQL Server (MSSQLSERVER)', N'SQL Server Agent (MSSQLSERVER)');
```

JUSTIFICACIÓN TÉCNICA

- ✓ Cuentas de servicio dedicadas: se definen cuentas locales (svcSQL y svcAgent) y el principio de mínimo privilegio.
- ✓ Automatización con xp_cmdshell: se usa sc config para asociar cuentas a los servicios.
- ✓ Verificación robusta del Agente: se comprueba la existencia del servicio antes de configurarlo.
- ✓ Validación posterior con DMV: se consulta sys.dm_server_services para auditar las cuentas actuales.

BUENAS PRÁCTICAS

- ★ Cuentas de servicio dedicadas por motor y agente.
- ★ Verificación antes de configurar (evita errores si el Agente no existe).
- ★ Habilitar/deshabilitar xp_cmdshell solo mientras se usa.
- ★ Auditoría con sys.dm_server_services al finalizar.

 ENUNCIADO DEL EJERCICIO

Implemente consultas administrativas que permitan verificar el estado de las instancias SQL Server relacionadas con GradosTítulos.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Estado de los servicios de la instancia actual
SELECT servicename AS Servicio, status_desc AS Estado,
       startup_type_desc AS TipoInicio, service_account AS Cuenta,
       process_id AS PID, last_startup_time AS UltimoInicio,
       is_clustered AS EsClusterizado
FROM sys.dm_server_services;
GO

-- 2. Información general de la instancia (recursos del sistema)
SELECT cpu_count AS NucleosLogicos,
       physical_memory_kb / 1024 AS MemoriaRAM_MB,
       sqlserver_start_time AS InicioSQLServer,
       virtual_machine_type_desc AS TipoVirtualizacion
FROM sys.dm_os_sys_info;
GO

-- 3. Estado específico de la base de datos objetivo
SELECT name AS BaseDeDatos, state_desc AS Estado,
       recovery_model_desc AS ModeloRecuperacion,
       user_access_desc AS AccesoUsuarios,
       is_auto_shrink_on AS AutoShrink,
       is_auto_close_on AS AutoClose
FROM sys.databases
WHERE name = N'GradosTítulos';
GO

-- 4. Sesiones activas actualmente en GradosTítulos
SELECT session_id, login_name, [status], host_name, program_name
FROM sys.dm_exec_sessions
WHERE database_id = DB_ID('GradosTítulos')
   AND is_user_process = 1;
```

 JUSTIFICACIÓN TÉCNICA

- ✓ sys.dm_server_services: muestra el estado, inicio y cuenta de los servicios de la instancia.
- ✓ sys.dm_os_sys_info: recursos del sistema (CPU, memoria, inicio de SQL Server).
- ✓ Verificación de la base de datos objetivo: estado y configuración de GradosTítulos.
- ✓ Sesiones activas: permite monitorear conexiones en tiempo real a la base de datos.

 BUENAS PRÁCTICAS

- ★ Monitoreo proactivo y centralizado.
- ★ Mínimo impacto: DMVs de solo lectura.
- ★ Enfoque en el estado de servicios y recursos.
- ★ Seguridad de acceso y mínimos privilegios.
- ★ Documentación de línea base.

 ENUNCIADO DEL EJERCICIO

Configure la instancia SQL Server para permitir únicamente conexiones seguras a la base de datos GradosTítulos.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Forzar el cifrado en todas las conexiones (Force Encryption)
EXEC xp_instance_regwrite
    N'HKEY_LOCAL_MACHINE',
    N'Software\Microsoft\MSSQLServer\MSSQLServer\SuperSocketNetLib',
    N'ForceEncryption',
    REG_DWORD,
    1;
GO

-- 2. Habilitar únicamente el protocolo TCP/IP
EXEC xp_instance_regwrite
    N'HKEY_LOCAL_MACHINE',
    N'Software\Microsoft\MSSQLServer\MSSQLServer\SuperSocketNetLib\Tcp',
    N'Enabled',
    REG_DWORD,
    1;
GO

-- 3. Deshabilitar Named Pipes (menos seguro)
EXEC xp_instance_regwrite
    N'HKEY_LOCAL_MACHINE',
    N'Software\Microsoft\MSSQLServer\MSSQLServer\SuperSocketNetLib\Np',
    N'Enabled',
    REG_DWORD,
    0;
GO

-- 4. Verificar la configuración actual (solo lectura)
SELECT registry_key, value_name, value_data
FROM sys.dm_server_registry
WHERE registry_key LIKE '%SuperSocketNetLib%'
AND value_name IN ('ForceEncryption', 'Enabled')
ORDER BY registry_key, value_name;
GO

-- 5. Aviso de reinicio necesario
PRINT 'Los cambios en los protocolos y cifrado requieren reiniciar
el servicio SQL Server para surtir efecto.';
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Forzar el cifrado (ForceEncryption): obliga a que todas las conexiones usen SSL/TLS.
- ✓ Solo TCP/IP habilitado: reduce la superficie de ataque y mejora la seguridad de la comunicación.
- ✓ xp_instance_regwrite modifica el Registro de la instancia actual de forma portable.
- ✓ Verificación con sys.dm_server_registry: confirma que los cambios se aplicaron.

 BUENAS PRÁCTICAS

- ★ Defensa en profundidad.
- ★ Deshabilitar protocolos no necesarios.
- ★ Configuración documentada y automatizada.
- ★ Reinicio controlado.
- ★ Complemento con firewall (puerto TCP 1433 y restricciones de IP).

1

PROYECTO 1 · TEMA 3 · ROLES

ROLES PARA EL PROCESO DE GRADOS Y TÍTULOS

ENUNCIADO DEL EJERCICIO

Diseñe roles personalizados para gestionar el acceso a los esquemas: Catalogo, Academico, Tramite, Evaluacion, Documento.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- Crear un rol por cada esquema con permisos de lectura y escritura completos
CREATE ROLE Rol_Catalogo_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Catalogo] TO Rol_Catalogo_Admin;

CREATE ROLE Rol_Academico_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Academico] TO Rol_Academico_Admin;

CREATE ROLE Rol_Tramite_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Tramite] TO Rol_Tramite_Admin;

CREATE ROLE Rol_Evaluacion_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Evaluacion] TO Rol_Evaluacion_Admin;

CREATE ROLE Rol_Documento_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO Rol_Documento_Admin;
GO

-- Verificar permisos concedidos (consulta a sys.database_permissions)
```

JUSTIFICACIÓN TÉCNICA

- ✓ Rol independiente para cada esquema.
- ✓ Permisos completos de lectura y escritura a nivel de esquema.
- ✓ Nuevos objetos heredan automáticamente los permisos.
- ✓ Verificación inmediata de los permisos.

BUENAS PRÁCTICAS

- ★ Principio de mínimo privilegio por área.
- ★ Escalabilidad y fácil mantenimiento.
- ★ Nombres descriptivos y coherentes.
- ★ Documentación y auditoría.

 ENUNCIADO DEL EJERCICIO

Los evaluadores deben consultar tesis y registrar observaciones sin modificar información académica.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- Crear el rol
CREATE ROLE Rol_EvaluadorTesis;

-- Lectura de información de tesis
GRANT SELECT ON [Tramite].[TrabajoInvestigacion] TO Rol_EvaluadorTesis;
GRANT SELECT ON [Tramite].[DesignacionJurado] TO Rol_EvaluadorTesis;
GRANT SELECT ON [Tramite].[V_EstadoTrabajo] TO Rol_EvaluadorTesis;
GRANT SELECT ON [Evaluacion].[V_NotaFinalSustentacion] TO Rol_EvaluadorTesis;

-- Registrar observaciones (INSERT y UPDATE)
GRANT INSERT, UPDATE ON [Evaluacion].[EvaluacionTrabajo] TO Rol_EvaluadorTesis;

-- Denegar modificaciones en otros esquemas
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Academico] TO Rol_EvaluadorTesis;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Catalogo] TO Rol_EvaluadorTesis;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO Rol_EvaluadorTesis;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Tramite] TO Rol_EvaluadorTesis;
GO

-- Verificar permisos del rol (consulta a sys.database_permissions)
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Acceso solo a información de tesis y sustentaciones.
- ✓ Puede registrar observaciones (INSERT y UPDATE en la tabla de evaluación).
- ✓ Negación explícita de modificaciones en otros esquemas.
- ✓ GRANT sobre objetos específicos prevalece sobre DENY de esquema.

 BUENAS PRÁCTICAS

- ★ Mínimo privilegio.
- ★ Separación de funciones.
- ★ Protección de información académica y sensible.
- ★ Auditoría y control.

 ENUNCIADO DEL EJERCICIO

Cree una jerarquía de roles para asignar tareas de operación diaria y supervisión del sistema.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- Crear roles
CREATE ROLE Rol_Operador;
CREATE ROLE Rol_Supervisor;

-- Rol_Operador: permisos operativos limitados
GRANT SELECT ON SCHEMA::[Catalogo] TO Rol_Operador;
GRANT SELECT, INSERT, UPDATE, DELETE ON [Tramite] TO Rol_Operador;
GRANT SELECT ON [Evaluacion] TO Rol_Operador;

-- Rol_Supervisor: incluye todo Rol_Operador + más permisos
GRANT SELECT, INSERT, UPDATE, DELETE ON [Documento] TO Rol_Supervisor;
GRANT SELECT, INSERT, UPDATE, DELETE ON [Pago] TO Rol_Supervisor;
GRANT DELETE ON [Tramite] TO Rol_Supervisor;

-- Asignación de jerarquía
EXEC sp_addrolemember 'Rol_Supervisor', 'Rol_Operador';
-- (Consulta a sys.database_role_members)

-- Verificar permisos por rol (consulta a sys.database_permissions)
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Jerarquía: Rol_Supervisor hereda todos los permisos de Rol_Operador.
- ✓ Facilita la administración y delegación de funciones.
- ✓ Control claro entre operación diaria y supervisión.

 BUENAS PRÁCTICAS

- ★ Jerarquía de roles para simplificar la gestión.
- ★ Principio de mínimo privilegio en cada nivel.
- ★ Denegación explícita de acceso a datos financieros para el operador.
- ★ Flexibilidad y escalabilidad en la administración.

1

PROYECTO 1
OPERADORES SOLO CONSULTAN ESTUDIANTES

ENUNCIADO DEL EJERCICIO

Los operadores pueden consultar datos de estudiantes, pero no modificarlos.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- Rol de consulta para operadores
CREATE ROLE Rol_OperadorConsulta;
GO

-- Permitir solo consulta de datos de estudiantes
GRANT SELECT ON [dbo].[Facultad] TO Rol_OperadorConsulta;
GRANT SELECT ON [dbo].[Matricula] TO Rol_OperadorConsulta;
GRANT SELECT ON [dbo].[Persona] TO Rol_OperadorConsulta;
GRANT SELECT ON [dbo].[ProgramaEstudios] TO Rol_OperadorConsulta;
GRANT SELECT ON [Tramite].[V_CondicionEgresado] TO Rol_OperadorConsulta;
GO

-- Denegar cualquier modificación en todos los esquemas
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Academico] TO Rol_OperadorConsulta;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Catalogo] TO Rol_OperadorConsulta;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO Rol_OperadorConsulta;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Evaluacion] TO Rol_OperadorConsulta;
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Tramite] TO Rol_OperadorConsulta;
GO

-- Verificar permisos asignados
SELECT p.name AS Rol,
       CASE perm.class WHEN 3 THEN SCHEMA_NAME(perm.major_id)
                     ELSE OBJECT_NAME(perm.major_id) END AS Objetivo,
       perm.permission_name AS Permiso, perm.state_desc AS Estado
FROM sys.database_permissions perm
JOIN sys.database_principals p ON perm.grantee_principal_id = p.principal_id
WHERE p.name = N'Rol_OperadorConsulta'
ORDER BY Estado, Objetivo;
```

Script reconstruido a partir de la tabla de asignación de permisos mostrada en el proyecto original.

JUSTIFICACIÓN TÉCNICA

- ✓ Se otorgan permisos SELECT sobre las tablas necesarias para consultar datos de estudiantes.
- ✓ Se niega explícitamente cualquier acción de modificación (INSERT, UPDATE, DELETE) en todos los esquemas.
- ✓ Se aplica el principio de mínimo privilegio para proteger la integridad de la información.

BUENAS PRÁCTICAS

- ★ Mínimo privilegio: solo se otorga lo estrictamente necesario.
- ★ Denegación explícita: refuerza la seguridad ante cambios futuros.
- ★ Granularidad por objeto: permisos específicos y controlados.
- ★ Segregación de funciones: evita conflictos de intereses.

2

PROYECTO 2

RESTRICCIÓN DE MODIFICACIÓN DE DOCUMENTOS OFICIALES

 ENUNCIADO DEL EJERCICIO

Solo la Oficina de Grados y Títulos puede modificar documentos oficiales.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Crear el rol para la oficina de grados y títulos
CREATE ROLE Rol_OficinaGradosTitulos;
GO

-- 2. Conceder control total sobre el esquema Documento
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Documento]
    TO Rol_OficinaGradosTitulos;

-- Permisos de lectura en esquemas relacionados
GRANT SELECT ON SCHEMA::[Tramite] TO Rol_OficinaGradosTitulos;
GRANT SELECT ON SCHEMA::[Academico] TO Rol_OficinaGradosTitulos;
GRANT SELECT ON SCHEMA::[Evaluacion] TO Rol_OficinaGradosTitulos;

-- 3. Denegar modificación a todos los demás roles personalizados
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Operador];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Supervisor];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Catalogo_Admin];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Academico_Admin];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Tramite_Admin];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Rol_Evaluacion_Admin];

-- 4. Denegar a usuarios que tienen rol de escritura general
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [Interno];
DENY INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO [ConsultorExterno];
GO
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Se concede control total sobre el esquema Documento únicamente al rol que representa a la Oficina de Grados y Títulos.
- ✓ A todos los demás roles y usuarios se les niega explícitamente cualquier tipo de modificación.
- ✓ Así se asegura que solo el personal autorizado pueda crear, editar o eliminar documentos oficiales.

 BUENAS PRÁCTICAS

- ★ Segregación de funciones: cada rol tiene responsabilidades claramente definidas.
- ★ Principio de mínimo privilegio: acceso solo a lo necesario.
- ★ Control centralizado: los cambios se administran en roles, no en usuarios.
- ★ Trazabilidad: fácil auditoría y cumplimiento normativo.

3

PROYECTO 3
ROLES ESPECÍFICOS POR ÁREA FUNCIONAL

ENUNCIADO DEL EJERCICIO

Cada área (académica, catálogo, trámite, evaluación) debe tener roles con permisos específicos.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- Roles y permisos por área funcional (resumen consolidado)
CREATE ROLE Rol_Academico_Admin;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::[Academico] TO Rol_Academico_Admin;

CREATE ROLE Rol_Catalogo_Admin;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::[Catalogo] TO Rol_Catalogo_Admin;

CREATE ROLE Rol_Tramite_Admin;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::[Tramite] TO Rol_Tramite_Admin;

CREATE ROLE Rol_Evaluacion_Admin;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::[Evaluacion] TO Rol_Evaluacion_Admin;

CREATE ROLE Rol_Documento_Admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::[Documento] TO Rol_Documento_Admin;

-- Consulta operativa (solo lectura controlada)
CREATE ROLE Rol_Operador;
GRANT SELECT ON [dbo].[VistasEspecificas] TO Rol_Operador;

-- Supervisión (lectura ampliada + reportes)
CREATE ROLE Rol_Supervisor;
GRANT SELECT ON SCHEMA::[Academico] TO Rol_Supervisor;
GRANT SELECT ON SCHEMA::[Tramite] TO Rol_Supervisor;
GO
```

Script consolidado a partir del resumen de roles y permisos por área mostrado en el proyecto original.

JUSTIFICACIÓN TÉCNICA

- ✓ Se crean roles por cada área funcional para aislar responsabilidades y otorgar únicamente los permisos necesarios.
- ✓ Esto facilita la administración, mejora la seguridad y reduce el riesgo de exposición de datos.

BUENAS PRÁCTICAS

- ★ Diseño por roles alineado a la estructura organizacional.
- ★ Mínimo privilegio: solo lo necesario para cada actividad.
- ★ Administración centralizada: fácil mantenimiento y escalabilidad.
- ★ Auditoría y cumplimiento: mejor visibilidad y control.

1

PROYECTO 1 · TEMA 5 · CIFRADO

IMPLEMENTACIÓN DE TDE PARA GRADOSTITULOS

 ENUNCIADO DEL EJERCICIO

Proteja los archivos físicos de la base de datos GradosTitulos mediante Transparent Data Encryption (TDE).

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 1 (TEMA 5): Habilitar TDE en GradosTitulos
USE master;
GO

-- 1. Clave maestra en master (si no existe)
IF NOT EXISTS (SELECT * FROM sys.symmetric_keys WHERE name = N'##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = N'P@ssw0rd_MasterKey_2026';
GO

-- 2. Certificado protegido por la clave maestra
CREATE CERTIFICATE CertificadoTDE_Grados
    WITH SUBJECT = N'Certificado para TDE de GradosTitulos';
GO

-- 3. Clave de cifrado de base de datos (DEK) en GradosTitulos
USE GradosTitulos;
GO
CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE CertificadoTDE_Grados;
GO

-- 4. Activar TDE
ALTER DATABASE GradosTitulos SET ENCRYPTION ON;
GO

-- 5. Verificar el estado del cifrado
SELECT DB_NAME(database_id) AS BaseDatos,
    encryption_state_desc AS EstadoCifrado,
    percent_complete AS PorcentajeCompletado,
    key_algorithm AS Algoritmo,
    key_length AS LongitudClave
FROM sys.dm_database_encryption_keys
WHERE database_id = DB_ID('GradosTitulos');
```

 JUSTIFICACIÓN TÉCNICA

- ✓ TDE cifra los archivos .mdf, .ndf y .ldf en reposo a nivel de página con AES-256.
- ✓ Protege los datos ante robo de disco o acceso a copias de seguridad sin el certificado y la clave maestra.
- ✓ El cifrado es transparente para las aplicaciones.

 BUENAS PRÁCTICAS

- ★ Respalidar el certificado y la clave maestra inmediatamente.
- ★ Usar AES-256, estándar seguro y recomendado.
- ★ Monitorear el progreso hasta que percent_complete sea 0.
- ★ Guardar los archivos de auditoría en una unidad separada.

 ENUNCIADO DEL EJERCICIO

Implemente Always Encrypted para proteger DNI, Correo electrónico y Teléfono de los estudiantes.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 2 (TEMA 5): Always Encrypted sobre Persona
USE GradosTitulos;
GO

-- 1. Clave maestra de columna (CMK)
IF NOT EXISTS (SELECT 1 FROM sys.column_master_keys WHERE name = N'CMK_GradosPersona')
CREATE COLUMN MASTER KEY CMK_GradosPersona
WITH (
    KEY_STORE_PROVIDER_NAME = N'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = N'CurrentUser/My/A1B2C3D4E5F6A7B8C9D0E1F2A3B4C5D6E7F8A9B0'
);
GO

-- 2. Clave de cifrado de columna (CEK)
IF NOT EXISTS (SELECT 1 FROM sys.column_encryption_keys WHERE name = N'CEK_Persona')
CREATE COLUMN ENCRYPTION KEY CEK_Persona
WITH VALUES (
    COLUMN_MASTER_KEY = CMK_GradosPersona,
    ALGORITHM = N'RSA_OAEP',
    ENCRYPTED_VALUE = 0x0123456789ABCDEF0123456789ABCDEF
);
GO

-- 3. Cifrar columnas en Academico.Persona
ALTER TABLE Academico.Persona ALTER COLUMN DNI CHAR(8) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = CEK_Persona,
        ENCRYPTION_TYPE = DETERMINISTIC, ALGORITHM = N'AEAD_AES_256_CBC_HMAC_SHA_256');

ALTER TABLE Academico.Persona ALTER COLUMN Correo VARCHAR(120) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = CEK_Persona,
        ENCRYPTION_TYPE = RANDOMIZED, ALGORITHM = N'AEAD_AES_256_CBC_HMAC_SHA_256');

ALTER TABLE Academico.Persona ALTER COLUMN Telefono VARCHAR(15) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = CEK_Persona,
        ENCRYPTION_TYPE = RANDOMIZED, ALGORITHM = N'AEAD_AES_256_CBC_HMAC_SHA_256');
GO

-- 4. Verificar columnas cifradas
SELECT t.name AS Tabla, c.name AS Columna,
    cek.name AS ClaveCifrada, ce.encryption_type_desc AS TipoCifrado
FROM sys.columns c
JOIN sys.tables t ON c.object_id = t.object_id
JOIN sys.column_encryption_keys cek ON c.column_encryption_key_id = cek.column_encryption_key_id
WHERE c.encryption_type IS NOT NULL AND t.name = 'Persona';
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Always Encrypted cifra los datos en el cliente antes de enviarlos al motor.
- ✓ El servidor nunca ve los valores en texto claro, mitigando ataques de administradores o malware.
- ✓ DNI usa cifrado determinista (búsquedas exactas) y Correo/Teléfono aleatorio (mayor seguridad).

 BUENAS PRÁCTICAS

- ★ CMK en almacén de certificados de Windows o Azure Key Vault.
- ★ Elegir tipo de cifrado según el uso: determinístico o aleatorio.
- ★ Interfazación BIN2 obligatoria en columnas cifradas.
- ★ Configurar cliente con Column Encryption Setting = Enabled.

 ENUNCIADO DEL EJERCICIO

Proteja la información personal de asesores y jurados de tesis.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- PROYECTO 3 (TEMA 5): Protección de datos de asesores y jurados (docentes)
-- La protección de Persona (Proyecto 2) cubre automáticamente a estos roles
USE GradosTitulos;
GO

-- 1. Consulta de docentes (asesores/jurados) - columnas cifradas
SELECT p.IdPersona, p.DNI, p.Nombres, p.ApellidoPaterno,
       d.GradoAcademico, d.TipoContrato
FROM Academico.Persona p
JOIN Academico.Docente d ON p.IdPersona = d.IdPersona
WHERE d.TipoPersona = 'DOCENTE' AND d.EstadoActivo = 1;

-- 2. Para ver datos en claro, el usuario/app debe tener acceso a la CMK
-- (se concede en el almacén de certificados / Azure Key Vault, no desde T-SQL)

-- 3. Verificación de datos cifrados de asesores y jurados
SELECT p.IdPersona, p.DNI, p.Correo, p.Telefono
FROM Academico.Persona p
JOIN Tramite.DesignacionAsesor da ON p.IdPersona = da.IdDocente
UNION
SELECT p.IdPersona, p.DNI, p.Correo, p.Telefono
FROM Academico.Persona p
JOIN Tramite.DesignacionJurado dj ON p.IdPersona = dj.IdDocente;
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Asesores y jurados son docentes, por lo que sus datos personales (DNI, Correo, Teléfono) residen en Persona y están protegidos por Always Encrypted.
- ✓ Sin la CMK, los valores se muestran cifrados (binarios), protegiendo la privacidad incluso de administradores.

 BUENAS PRÁCTICAS

- ★ Reutilizar la protección existente (Persona), no duplicar esfuerzos.
- ★ Conceder acceso a la CMK solo a usuarios y aplicaciones autorizadas.
- ★ Los datos nunca se exponen en texto claro dentro del servidor.
- ★ Cumplir normativas de protección de datos personales.

ENUNCIADO DEL EJERCICIO

Registrar todos los accesos realizados a la base de datos GradosTitulos.

SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Crear auditoría de servidor
USE master;
GO
CREATE SERVER AUDIT [Audit_Accesos_Grados]
    TO FILE (FILEPATH = N'C:\SQLAudit\', MAXSIZE = 100 MB,
            MAX_ROLLOVER_FILES = 5, RESERVE_DISK_SPACE = OFF)
    WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT [Audit_Accesos_Grados] WITH (STATE = ON);
GO

-- 2. Especificación de auditoría en GradosTitulos
USE GradosTitulos;
GO
CREATE DATABASE AUDIT SPECIFICATION [Audit_Accesos_BD]
    FOR SERVER AUDIT [Audit_Accesos_Grados]
    ADD (SCHEMA_OBJECT_ACCESS_GROUP),
    ADD (DATABASE_PRINCIPAL_IMPERSONATION_GROUP)
    WITH (STATE = ON);
GO

-- 3. Operaciones de prueba
SELECT TOP 5 * FROM Academico.Persona;
EXEC Tramite.SP_RegistrarTramite @IdMatricula = 1,
    @IdTipoTramite = 2, @IdCoordinacion = 1, @IdTramite = 0;
GO

-- 4. Consultar eventos capturados
SELECT event_time AS FechaHora, session_server_principal_name AS Usuario,
    database_name AS BaseDatos, object_name AS Objeto, statement AS Sentencia,
    action_id AS IdAccion, succeeded AS Exito, additional_information AS InfoAdicional
FROM sys.fn_get_audit_file('C:\SQLAudit\*', DEFAULT, DEFAULT)
WHERE database_name = N'GradosTitulos'
ORDER BY event_time DESC;
```

JUSTIFICACIÓN TÉCNICA

- ✓ SQL Server Audit es nativo, ligero y centralizado.
- ✓ Se usa SCHEMA_OBJECT_ACCESS_GROUP para capturar todas las operaciones de acceso.
- ✓ DATABASE_PRINCIPAL_IMPERSONATION_GROUP registra conexiones y cambios de contexto.
- ✓ La auditoría se almacena en archivos con rotación controlada.

BUENAS PRÁCTICAS

- ★ Mínimo privilegio: auditamos solo la base de datos objetivo.
- ★ Retención controlada: tamaño máximo y número de archivos rotativos.
- ★ ON_FAILURE = CONTINUE evita que la auditoría detenga el servidor.
- ★ Separación de destinos: guardar en unidad y archivar los eventos capturados.
- ★ Verificación periódica: revisar y archivar los eventos capturados.

 ENUNCIADO DEL EJERCICIO

Registrar todas las operaciones INSERT, UPDATE y DELETE realizadas sobre las tablas del esquema Tramite.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Crear auditoría de servidor
USE master;
GO
CREATE SERVER AUDIT [Audit_Tramite]
    TO FILE (FILEPATH = N'C:\SQLAudit\', MAXSIZE = 200 MB,
            MAX_ROLLOVER_FILES = 10, RESERVE_DISK_SPACE = OFF)
    WITH (ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT [Audit_Tramite] WITH (STATE = ON);
GO

-- 2. Especificación de auditoría para esquema Tramite
USE GradosTitulos;
GO
CREATE DATABASE AUDIT SPECIFICATION [Audit_Modif_Tramite]
    FOR SERVER AUDIT [Audit_Tramite]
    ADD (INSERT ON SCHEMA::[Tramite] BY [public]),
    ADD (UPDATE ON SCHEMA::[Tramite] BY [public]),
    ADD (DELETE ON SCHEMA::[Tramite] BY [public])
    WITH (STATE = ON);
GO

-- 3. Operaciones de prueba
INSERT INTO Tramite.CoordinacionGT (IdFacultad) VALUES (1);
INSERT INTO Tramite.Tramite (IdMatricula, IdTipoTramite, IdEstadoTramite, IdCoordinacion)
    VALUES (1, 1, 1, 1);
UPDATE Tramite.Tramite SET Observaciones = 'Prueba auditoría' WHERE IdTramite = 1;
DELETE FROM Tramite.Tramite WHERE IdTramite = 1;
GO

-- 4. Consultar eventos capturados
SELECT event_time AS FechaHora, session_server_principal_name AS Usuario,
       schema_name AS Esquema, object_name AS Objeto, action_id AS Accion,
       statement AS Sentencia, succeeded AS Exito
FROM sys.fn_get_audit_file('C:\SQLAudit\', DEFAULT, DEFAULT)
WHERE schema_name = N'Tramite' AND action_id IN ('IN', 'UP', 'DL')
ORDER BY event_time DESC;
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Solo se registran operaciones que modifican datos (INSERT, UPDATE, DELETE).
- ✓ Esto reduce el volumen de eventos y mejora el rendimiento.
- ✓ La auditoría está enfocada y es más fácil de analizar.

 BUENAS PRÁCTICAS

- ★ Solo a nivel de esquema para cubrir tablas futuras automáticamente.
- ★ Auditoría dedicada para facilitar consultas y mantenimiento.
- ★ Filtro en la consulta por acción para obtener solo lo relevante.

 ENUNCIADO DEL EJERCICIO

Diseñe una solución de auditoría que registre: inicio de sesión, cambios de permisos, creación de usuarios, eliminación de objetos y modificaciones de datos críticos.

 SCRIPT DE LA SOLUCIÓN EN T-SQL

```
-- 1. Crear auditoría de servidor centralizada
USE master;
GO
CREATE SERVER AUDIT [Audit_Institucional]
    TO FILE (FILEPATH = N'C:\SQLAudit\', MAXSIZE = 500 MB,
            MAX_ROLLOVER_FILES = 20, RESERVE_DISK_SPACE = OFF)
    WITH (ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT [Audit_Institucional] WITH (STATE = ON);
GO

-- 2. Especificación de auditoría a nivel de servidor
CREATE SERVER AUDIT SPECIFICATION [Audit_Servidor_Global]
    FOR SERVER AUDIT [Audit_Institucional]
    ADD (SUCCESSFUL_LOGIN_GROUP),
    ADD (FAILED_LOGIN_GROUP),
    ADD (SERVER_PERMISSION_CHANGE_GROUP),
    ADD (SERVER_PRINCIPAL_CHANGE_GROUP),
    ADD (DATABASE_CHANGE_GROUP)
    WITH (STATE = ON);
GO

-- 3. Especificación de auditoría a nivel de base de datos
USE GradosTitulos;
GO
CREATE DATABASE AUDIT SPECIFICATION [Audit_BD_Institucional]
    FOR SERVER AUDIT [Audit_Institucional]
    ADD (DATABASE_PERMISSION_CHANGE_GROUP),
    ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),
    ADD (DATABASE_OBJECT_CHANGE_GROUP),
    ADD (INSERT, UPDATE, DELETE ON SCHEMA::[Tramite] BY [public]),
    ADD (INSERT, UPDATE, DELETE ON [Tramite].[Pago] BY [public])
    WITH (STATE = ON);
GO

-- 4. Operaciones de prueba (simulación)
SELECT * FROM Academico.Persona;
CREATE USER TempUser WITHOUT LOGIN;
CREATE TABLE TestDrop (Id INT);
DROP TABLE TestDrop;
INSERT INTO Tramite.Tramite (IdMatricula, IdTipoTramite, IdEstadoTramite, IdCoordinacion)
VALUES (1, 1, 1, 1);
GO

-- 5. Consultar todos los eventos capturados
SELECT event_time AS FechaHora, database_name AS BaseDatos,
       object_name AS Objeto, schema_name AS Esquema, action_id AS IdAccion,
       class_type_desc AS TipoClase, succeeded AS Exito
FROM sys.fn_get_audit_file('C:\SQLAudit\*', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
```

 JUSTIFICACIÓN TÉCNICA

- ✓ Se implementa una solución de auditoría combinando auditoría de servidor (logins y permisos principales) y de base de datos (permisos, principales, objetos y datos críticos).
- ✓ Esto garantiza trazabilidad completa de actividades relevantes.

 BUENAS PRÁCTICAS

- ★ Centralización: un solo Server Audit para toda la institución.
- ★ Separación lógica: especificaciones independientes a nivel de servidor y de BD.
- ★ Solo eventos relevantes: se auditan las acciones más críticas.
- ★ Enfoque en el esquema Tramite y Pago por ser los más críticos.
- ★ Revisión forense: consultas y reportes para detectar accesos sospechosos.